

# AWS CDK, Serverless & CloudFormation for Node.js Microservices

---

Build, Deploy & Scale Production-Ready Node.js & NestJS  
Microservices on AWS



# Agenda

1. Introduction to Infrastructure as Code and Microservices
2. AWS CDK Fundamentals
3. Microservices Deployment with CDK
4. NestJS on AWS Lambda
5. Serverless Framework Overview
6. CloudFormation Basics
7. Terraform and IaC Comparison
8. Sample Architecture and CI/CD
9. Best Practices and Production Readiness
10. Next Steps and Advanced Topics

# Introduction to Infrastructure as Code and Microservices

# What You Will Learn

## Modern IaC for Node.js Microservices

Automate cloud deployments using code

## Build Scalable Architectures

Design for production-readiness and growth

## Compare Key IaC Tools

AWS CDK, Serverless, CloudFormation, Terraform

## Deploy NestJS on AWS Lambda

Run robust Node.js backends serverlessly



# What is Infrastructure as Code (IaC)?

## Manual Cloud Setup: Not Scalable

Manual provisioning is error-prone and slow.

## Key Benefits of IaC

Ensures repeatability, version control, and automation.

## Infrastructure as Code (IaC)

Define and manage cloud resources using code.

## Popular IaC Tools

CloudFormation, Terraform, AWS CDK, Serverless Framework.



# Microservices on AWS



## Microservices: Independently Deployable

Each service can be updated and scaled on its own

## Service-Specific Infrastructure

Teams manage resources tailored to each microservice

## Event & API-Driven Communication

Services interact via events or APIs for flexibility

## AWS: Natural Fit for Microservices

AWS offers tools for modular, scalable architectures

---



# AWS Services Used

## Compute: AWS Lambda

Run code without managing servers

## API Management: API Gateway

Expose RESTful and HTTP APIs for services

## Storage: DynamoDB & RDS

NoSQL and relational database options

## Messaging: SQS, SNS, EventBridge

Enable event-driven communication

## Security: IAM

Control access and permissions



# Serverless vs Traditional Hosting

## Eliminate Server Management

Focus on code, not infrastructure maintenance

## Pay Only for What You Use

Billing is based on actual executions

## Effortless Auto Scaling

Resources scale automatically with demand

## Accelerated Deployments

Rapid release cycles and updates



# AWS CDK Fundamentals



# What is AWS CDK?

## Code Your Cloud Infrastructure

Define AWS resources using familiar programming languages

## Supports TypeScript, JavaScript, Python

Choose your preferred language for infrastructure code

## Seamless CloudFormation Integration

CDK compiles your code into CloudFormation templates

---



# Why AWS CDK Over CloudFormation?

## Streamlined Infrastructure Code

Reduce repetitive CloudFormation boilerplate with CDK.

## Reusable & Modular Constructs

Create and share custom infrastructure components easily.

## Type Safety & Developer Tools

Leverage TypeScript/JavaScript for error checking and IDE support.

## Developer-Friendly Experience

Write, test, and maintain infrastructure like application code.



# CDK Project Structure



## Core Files in a CDK Project

- `cdk.json` configures the CDK CLI and project settings.
- `lib/stack.ts` defines your AWS resources and infrastructure.
- `bin/app.ts` is the entry point for your CDK application.



## Constructs & Stacks

- Constructs are reusable building blocks for AWS resources.
- Stacks group related resources for deployment and management.

# First CDK Deployment

## Initialize Your CDK Project

Start with 'cdk init app' to scaffold your project.

## Bootstrap Your AWS Environment

Run 'cdk bootstrap' to set up resources for deployment.

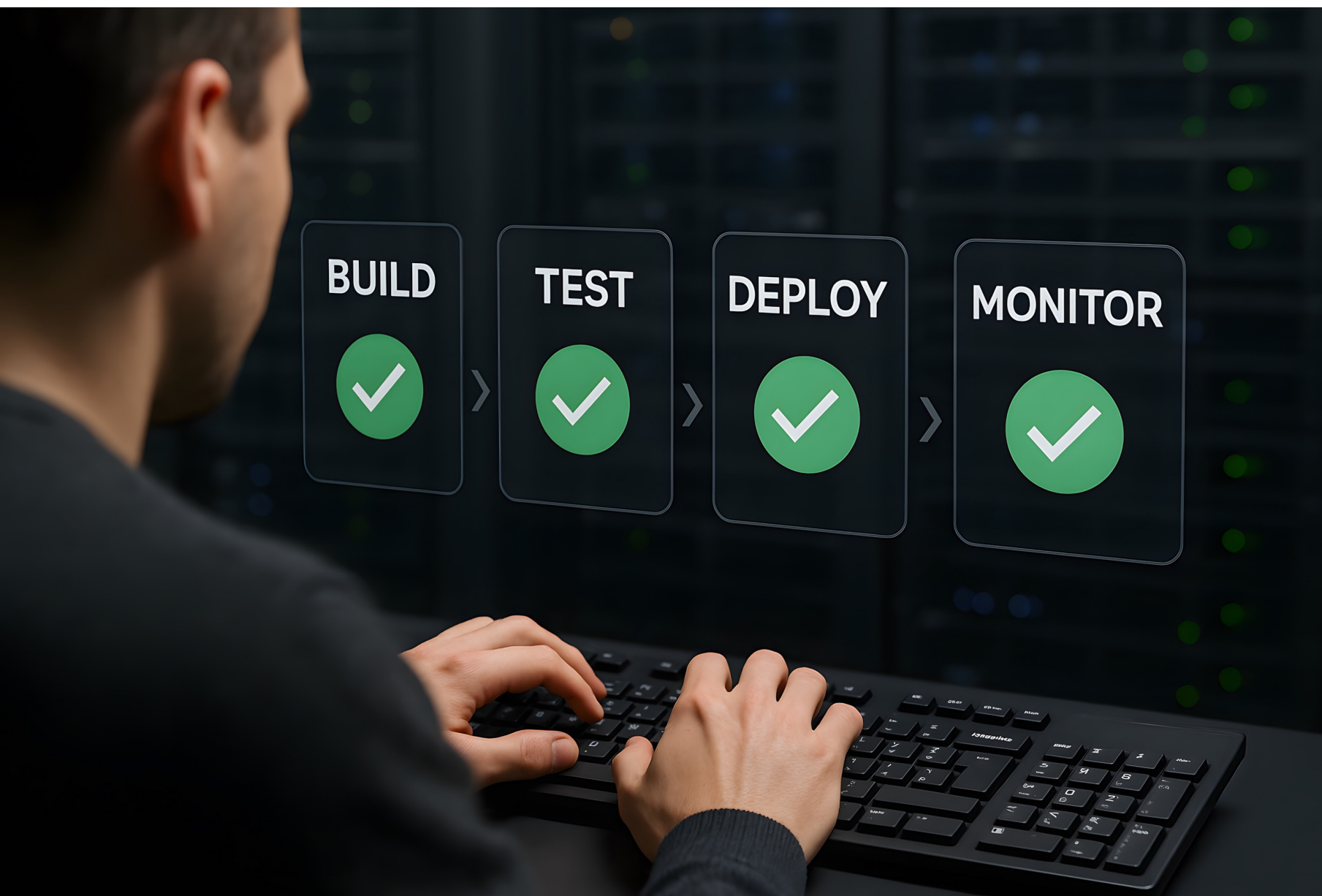
## Deploy Your Stack

Use 'cdk deploy' to launch infrastructure defined in code.

## Understand Stack Lifecycle

Stacks can be created, updated, or deleted; changes are managed automatically.

---





# Microservices Deployment with CDK

# Microservices with CDK

## Isolate Each Microservice

Deploy every microservice in its own CDK stack for autonomy.

## Share Common Infrastructure

Reuse constructs for databases, messaging, or networking across services.

## Enable Independent Deployments

Update, scale, or rollback services without affecting others.



# Deploying Node.js Lambda with CDK

## NodejsFunction: Simplified Lambda Deployment

Use NodejsFunction to define and deploy Node.js Lambdas efficiently.

## Efficient Bundling with esbuild

Leverage esbuild for fast, optimized Lambda packaging.

## Configuring Environment Variables

Pass runtime settings securely to your Lambda functions.

## Fine-Grained IAM Permissions

Assign least-privilege roles for secure Lambda execution.

# API Gateway + Lambda



## REST API vs HTTP API

Choose REST for advanced features; HTTP for lower latency



## Service-Specific Routing

Map API Gateway routes to individual microservices



## API Versioning Strategies

Use path or header-based versioning for smooth upgrades



# NestJS on AWS Lambda

# NestJS on AWS Lambda

## NestJS as a Modern Backend

Leverages TypeScript for scalable, maintainable APIs

## Adapting HTTP Server to Lambda

Wraps NestJS app for AWS Lambda event handling

## Cold Start Considerations

Optimize bundle size and memory to reduce startup latency

# NestJS Lambda Architecture



API Gateway routes requests to Lambda

- Acts as the entry point for HTTP traffic

Lambda executes NestJS application logic

- Handles business logic and responses

## Single vs Multiple Lambda Functions

- Single Lambda: easier management, shared resources
- Multiple Lambdas: fine-grained scaling, isolation

## Monorepo vs Single Service Deployment

- Monorepo: unified codebase, shared dependencies
  - Single service: focused deployments, simpler CI/CD
-



# CDK Stack for NestJS Lambda

## Build and Package NestJS Application

Compile NestJS app for Lambda deployment

## Efficient Bundling with esbuild

Optimize output for fast cold starts

## Deploy Using NodejsFunction Construct

Integrate Lambda with CDK for seamless deployment

## Configure Memory and Timeout Settings

Tune resources for performance and cost

# Environment Management

## Environment Variables: .env Files

Simple local config, not ideal for cloud secrets

## AWS Parameter Store & Secrets Manager

Secure, centralized management for sensitive data

## Stage-Based Configuration

Separate settings for dev, test, prod environments

# Serverless Framework Overview



# What is Serverless Framework?



## Command-Line Driven Deployment

Manage cloud resources using simple CLI commands

## Declarative YAML Configuration

Define services, functions, and events in easy-to-read YAML files

## Multi-Cloud Provider Support

Works seamlessly with AWS, Azure, and Google Cloud

---

# serverless.yml Explained

## Define Your Service

Set the name and purpose of your serverless project.

## Configure Provider Settings

Specify cloud provider, runtime, and region details.

## Declare Functions & Events

List Lambda functions and their triggering events.

## Add Custom Resources

Define additional AWS resources needed by your app.

# Deploy Node.js Lambda using SLS



## Effortless Deployment

Use 'serverless deploy' to launch Lambda functions.



## Clean Removal

Remove resources with 'serverless remove' for easy cleanup.

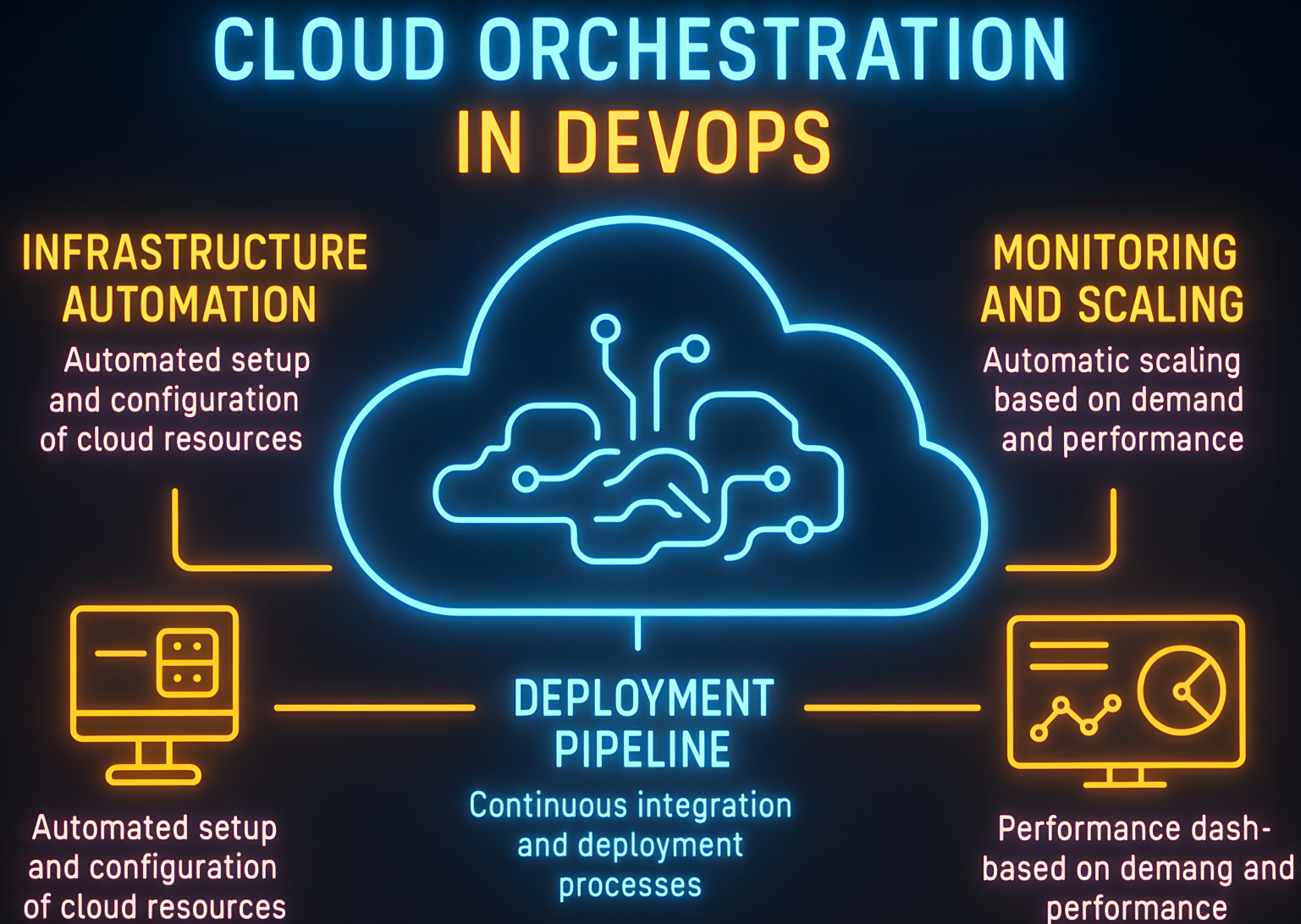


## Integrated Monitoring

Access logs and monitor Lambda execution directly via CLI.



# SLS vs AWS CDK



## CDK: Code-Driven Infrastructure

- Define resources using TypeScript, JavaScript, or Python
- Ideal for complex logic and reusable constructs

## SLS: Configuration-Driven Simplicity

- Use YAML files for quick, declarative setups
- Great for rapid prototyping and multi-cloud support

## Choosing the Right Tool

- CDK suits developers needing flexibility and code reuse
  - SLS fits teams preferring simple, config-first deployments
-

# CloudFormation Basics



# What is CloudFormation?

## AWS CloudFormation: Native IaC Solution

Automates AWS resource provisioning using code

## Template-Driven Infrastructure

Uses JSON or YAML files to define resources

## Declarative Approach

Describe desired state, CloudFormation handles deployment

---





# CloudFormation Template Anatomy



## Parameters: Dynamic Input Values

Enable template customization for different environments

## Resources: Core Infrastructure

Define AWS services like Lambda, API Gateway, DynamoDB

## Outputs: Key Information Sharing

Expose resource attributes (e.g., API endpoints) post-deployment

## Mappings: Conditional Logic

Map values for region, environment, or AMI selection

---



# CloudFormation Stack Lifecycle

## Stack Creation

Provision resources from template definitions.

## Updating Stacks

Modify resources by updating the template.

## Stack Deletion

Remove all resources managed by the stack.

## Automatic Rollbacks

Revert changes if deployment fails.

---



# Terraform and IaC Comparison



# What is Terraform?



## Provider-Based Infrastructure as Code

Define and manage resources across cloud providers using code.



## Multi-Cloud Flexibility

Supports AWS, Azure, GCP, and more for seamless deployments.



## Robust State Management

Tracks infrastructure changes and enables safe updates.

# Terraform vs AWS CDK vs CloudFormation



## CloudFormation: AWS-native control

- Deep integration with AWS services
- Declarative templates in JSON/YAML



## CDK: Developer-friendly AWS

- Use TypeScript, Python, or JavaScript for infra
- Reusable constructs and less boilerplate



## Terraform: Multi-cloud infra

- Supports AWS, Azure, GCP, and more
- Provider-based, state management built-in

# Sample Architecture and CI/CD



# Sample Architecture

## API Gateway as Entry Point

Centralizes routing for all microservices

## Multiple Lambda Functions

Each microservice runs independently and scales automatically

## EventBridge for Event-Driven Flow

Enables asynchronous communication between services

## DynamoDB for Scalable Storage

Provides fast, serverless data persistence

## CI/CD Pipeline Integration

Automates deployment and promotes rapid iteration

# CI/CD for Infrastructure



## Automated Deployments with GitHub Actions

Trigger infrastructure updates on code changes



## CDK Deploy Pipelines

Streamline provisioning and updates using CI/CD workflows



## Environment Promotion Strategies

Safely move changes from dev to staging to production

# Best Practices and Production Readiness



# Best Practices

## Isolate Each Microservice

Deploy every service in its own stack for clarity and control.

## Enforce Least Privilege IAM

Grant only necessary permissions to each resource.

## Implement Monitoring & Logging

Track metrics and logs for visibility and troubleshooting.

## Optimize for Cost

Use serverless and right-size resources to minimize expenses.

# What You Can Build Now

## Launch Production-Ready Microservices

Design, deploy, and scale Node.js services on AWS

## Automate Deployments End-to-End

Integrate CI/CD for seamless infrastructure updates

## Build Serverless NestJS Applications

Leverage NestJS with AWS Lambda for efficient backends

## Scale Cloud Systems Effortlessly

Utilize AWS services for robust, scalable architectures

# Next Steps and Advanced Topics



# Next Steps



## Master Advanced CDK Patterns

Explore reusable constructs and modular stack designs



## Build Event-Driven Architectures

Leverage AWS EventBridge, SQS, and SNS for microservice communication



## Implement Observability & Tracing

Integrate logging, monitoring, and distributed tracing for production systems